МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №5
Варіант 4

на тему «Ліниві обчислення: генератори та itertools»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. зрозуміти генератори, yield / yield from, ітератори та лінивість; навчитися працювати з нескінченними послідовностями; освоїти islice, takewhile, dropwhile, accumulate, count, repeat.

Теоретичні відомості

Ітератор — об'єкт зі станом, що повертає наступне значення і завершується StopIteration. **Лінивість**: значення обчислюються лише на запит, що економить пам'ять і дозволяє працювати з нескінченними джерелами.

Ліниві послідовності створюються генераторними виразами або функціями з **yield**; **yield from** делегує генерацію іншому ітератору. Генератор вичерпується після одного проходу.

Модуль **itertools**: islice / takewhile / dropwhile — обмеження й відбір; accumulate — префіксні суми, добутки та інші згортки; count, cycle, repeat — нескінченні джерела. Нескінченний потік завжди потрібно обмежувати (take, islice, takewhile).

Індивідуальне завдання (варіант 4)

Тема «Геометрична прогресія». Вхід: початкове значення q0 і множник r. Створити нескінченний генератор; взяти значення до порога та обчислити накопичений добуток / суму. Інструменти: yield, takewhile, accumulate.

Хід роботи

1. Нескінченний генератор прогресії

Прогресію реалізовано трьома способами: власний генератор з yield, через accumulate(repeat(r), mul, initial=q0) і через count. Усі три дають однаковий результат.

```
def geometric(q0: float, r: float) -> Iterator[float]:
    """Нескінченний генератор q0, q0*r, q0*r^2, ... (власний yield)."""
    value = q0
    while True:
        yield value
        value *= r
def geometric_accumulate(q0: float, r: float) -> Iterator[float]:
    """Та сама прогресія через itertools: accumulate(repeat(r), mul, initial=q0)."""
    return accumulate(repeat(r), operator.mul, initial=q0)

def geometric_count(q0: float, r: float) -> Iterator[float]:
    """Та сама прогресія через count: q0 * r**n для n = 0, 1, 2, ..."""
    return (q0 * r**n for n in count())
```

Лістинг 1 — фрагмент файлу streams.py

2. Обмеження потоку та накопичення

takewhile зупиняє нескінченний потік на порозі, dropwhile пропускає малі початкові члени, accumulate з operator.add / operator.mul дає накопичені суму та добуток. Для перевірки використано формулу суми S(n) = q0·(1 − r^n)/(1 − r).

```
def up_to_threshold(it: Iterable[float], threshold: float) -> Iterator[float]:
    """Брати значення, поки |x| <= threshold (takewhile): зупиняє потік."""
    return takewhile(lambda x: abs(x) <= threshold, it)

def running_sum(it: Iterable[Number]) -> Iterator[Number]:
    """Накопичена сума S1, S2, ... (accumulate з operator.add)."""
    return accumulate(it, operator.add)

def running_product(it: Iterable[Number]) -> Iterator[Number]:
    """Накопичений добуток P1, P2, ... (accumulate з operator.mul)."""
    return accumulate(it, operator.mul)
def geometric_sum_formula(q0: float, r: float, n: int) -> float:
    """Сума перших n членів за формулою S_n = q0 * (1 - r^n) / (1 - r)."""
    if n < 0:
        raise ValueError("n must be non-negative")
    if r == 1:
        return q0 * n
    return q0 * (1 - r**n) / (1 - r)
```

Лістинг 2 — фрагмент файлу streams.py

3. Універсальні утиліти

take, drop і sliding_window (на deque(maxlen=k)) не матеріалізують потік. Некоректний розмір вікна викликає ValueError одразу, а не при першому next().

```
def take(n: int, it: Iterable[T]) -> list[T]:
    """Перші n елементів (фінальна матеріалізація)."""
    if n < 0:
        raise ValueError("n must be non-negative")
    return list(islice(it, n))
def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    """Пропустити перші n елементів, повернути ітератор на решту (ліниво)."""
    if n < 0:
        raise ValueError("n must be non-negative")
    iterator = iter(it)
    next(islice(iterator, n, n), None)  # «промотати» n елементів
    return iterator

def sliding_window(it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    """Ковзні вікна розміру k через deque(maxlen=k) — без матеріалізації потоку.

    Перевірка k виконується ОДРАЗУ (а не при першому next), тому функція —
    звичайна, а генератор — внутрішній.
    """
    if k <= 0:
        raise ValueError("window size must be positive")

    def gen() -> Iterator[tuple[T, ...]]:
        iterator = iter(it)
        window: deque[T] = deque(islice(iterator, k), maxlen=k)
        if len(window) < k:
            return  # елементів менше за k — жодного вікна
        yield tuple(window)
        for x in iterator:
```

```
            window.append(x)
            yield tuple(window)

    return gen()
```

Лістинг 3 — фрагмент файлу streams.py

4. Тест лінивості

Обгортка counting рахує звернення до потоку; тест підтверджує, що обчислено рівно стільки елементів, скільки запитано. Усього 22 тести, зокрема крайні випадки: порожнє джерело, n = 0, k > довжини, k ≤ 0.

```
def test_geometric_is_lazy() -> None:
    calls = {"n": 0}

    def inc() -> None:
        calls["n"] += 1
    stream = counting(geometric(1, 2), inc)
    assert calls["n"] == 0  # нічого не обчислено до запиту
    assert take(5, stream) == [1, 2, 4, 8, 16]
    assert calls["n"] == 5  # рівно стільки, скільки запросили

    calls["n"] = 0
    list(up_to_threshold(counting(geometric(1, 2), inc), 10))
    assert calls["n"] == 5  # 1, 2, 4, 8 і перший «зайвий» 16, який зупинив takewhile
```

Лістинг 4 — фрагмент файлу tests/test_streams.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
=== Геометрична прогресія q0=3.0, r=2.0 ===
Перші 8 членів (yield)      : [3.0, 6.0, 12.0, 24.0, 48.0, 96.0, 192.0, 384.0]
Перші 8 (accumulate+repeat) : [3.0, 6.0, 12.0, 24.0, 48.0, 96.0, 192.0, 384.0]
Перші 8 (count)             : [3.0, 6.0, 12.0, 24.0, 48.0, 96.0, 192.0, 384.0]
Члени до порога 1000.0 (takewhile):
   [3.0, 6.0, 12.0, 24.0, 48.0, 96.0, 192.0, 384.0, 768.0]
Накопичена сума   (accumulate add): [3.0, 9.0, 21.0, 45.0, 93.0, 189.0, 381.0, 765.0,
    1533.0]
Накопичений добуток (accumulate mul): [3.0, 18.0, 216.0, 5184.0, 248832.0, 23887872.0,
    4586471424.0, 1761205026816.0, 1352605460594688.0]
Перевірка формулою S_n: 1533.0 == 1533.0
Відношення сусідніх членів: [2.0, 2.0, 2.0, 2.0, 2.0]

Таблиця (n, член, сума, добуток):
   1      3.0      3.0            3.0
   2      6.0      9.0           18.0
   3     12.0     21.0          216.0
   4     24.0     45.0         5184.0
   5     48.0     93.0       248832.0
   6     96.0    189.0     23887872.0

Спадна прогресія 100 * 0.5^n, поки член >= 1:
   [100.0, 50.0, 25.0, 12.5, 6.25, 3.125, 1.5625]
Пропустити члени < 50 (dropwhile), взяти 4: [81.0, 243.0, 729.0, 2187.0]
drop(3) + take(4): [1000.0, 10000.0, 100000.0, 1000000.0]
Ковзні вікна k=3: [(1.0, 2.0, 4.0), (2.0, 4.0, 8.0), (4.0, 8.0, 16.0)]
Рухоме середнє k=2: [3.0, 6.0, 12.0, 24.0, 48.0]
=== Базові потоки ===
fib_stream: drop 5, take 8 : [5, 8, 13, 21, 34, 55, 89, 144]
primes, перші 10           : [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
вікна сусідніх простих     : [(2, 3), (3, 5), (5, 7)]
кратні 3 -> суми вікон по 4: [30, 42, 54, 66, 78, 90, 102, 114, 126, 138]
chunked(range(10), 4)      : [(0, 1, 2, 3), (4, 5, 6, 7), (8, 9)]
islice нескінченного потоку: [1, -2, 4, -8, 16, -32]
```

Рисунок 1 — результат роботи програми (Lab_05)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_05
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 22 items
tests/test_streams.py::test_geometric_first_terms PASSED                 [  4%]
tests/test_streams.py::test_three_implementations_agree PASSED           [  9%]
tests/test_streams.py::test_up_to_threshold_stops_infinite_stream PASSED [ 13%]
tests/test_streams.py::test_decreasing_progression_while_above PASSED    [ 18%]
tests/test_streams.py::test_skip_below PASSED                            [ 22%]
tests/test_streams.py::test_running_sum_and_product PASSED               [ 27%]
tests/test_streams.py::test_sum_formula_matches_accumulate PASSED        [ 31%]
tests/test_streams.py::test_ratios_are_constant PASSED                   [ 36%]
tests/test_streams.py::test_progression_report PASSED                    [ 40%]
tests/test_streams.py::test_geometric_is_lazy PASSED                     [ 45%]
tests/test_streams.py::test_take_drop_on_count PASSED                    [ 50%]
tests/test_streams.py::test_drop_then_take PASSED                        [ 54%]
tests/test_streams.py::test_take_drop_edge_cases PASSED                  [ 59%]
tests/test_streams.py::test_windows_basic PASSED                         [ 63%]
tests/test_streams.py::test_window_edge_cases PASSED                     [ 68%]
tests/test_streams.py::test_moving_average PASSED                        [ 72%]
tests/test_streams.py::test_fib_stream PASSED                            [ 77%]
tests/test_streams.py::test_naturals_and_pipeline PASSED                 [ 81%]
tests/test_streams.py::test_primes PASSED                                [ 86%]
tests/test_streams.py::test_chunked PASSED                               [ 90%]
tests/test_streams.py::test_tee_safe_independent_copies PASSED           [ 95%]
tests/test_streams.py::test_accumulate_examples PASSED                   [100%]
============================== 22 passed in 0.07s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_05)

Висновок

Реалізовано ліниву обробку геометричної прогресії: нескінченний генератор із yield, обмеження порогом через takewhile, накопичені сума й добуток через accumulate. Додатково реалізовано take, drop, sliding_window, moving_average, chunked, fib_stream і потік простих чисел. Нескінченні потоки ніде не матеріалізуються повністю — це підтверджує тест лінивості. Крайні випадки оброблено, код проходить pytest, mypy, black і ruff.